

Instituto Tecnológico y de Estudios Superiores de Monterrey

Desarrollo de aplicaciones avanzadas de ciencias computacionales Gpo 505

Elda Guadalupe Quiroga González

Babyduck Entrega #0

Gabriel Eduardo Diaz Roa A00833574

22 de abril del 2025

1. Expresiones Regulares

Elemento léxico	Token	Expresión Regular
id	ID	[a-zA-Z_][a-zA-Z0-9_]*
Números Enteros	INT	[0-9]+
Números Flotantes	FLOAT	[0-9]+\.[0-9]+
Strings	STRING	\".*?\"
+	SUM	\+
-	MINUS	\-
*	MULTI	*
/	DIV	\/
<	L_THAN	<
>	G_THAN	>
!=	N_EQUAL	!=
=	ASSIGN	=
;	SEMI_COL	;
:	COLON	:
,	COMMA	,
(	L_PAREN	(
)	R_PAREN	)
{	L_BRACE	{
}	R_BRACE	}
[	L_BRACKET	[
]	R_BRACKET	]
program	PROGRAM	program
main	MAIN	main
end	END	end
var	VAR	var
print	PRINT	print
if	IF	if
else	ELSE	else
while	WHILE	While
do	DO	do

2. Tokens

Palabras clave:

program, main, end, print, if, else, while, do, var

Identificadores y constantes:

id, int, float, string

Operadores:

Aritméticos: +, -, *, /

Relacionales: <, >, !=

Asignación: =

Símbolos y signos de puntuación

Semicolon ;, Colon :, Comma ,

Paréntesis: L_PAREN (, R_PAREN)

Llaves: L_BRACE {, R_BRACE }

Corchetes: L_BRACKET [, R_BRACKET]

3. Context Free Grammar

programa $\rightarrow$ 'program' ID ';' vars funcs 'main' body 'end'

vars $\rightarrow$ var_decl vars

vars $\rightarrow \epsilon$

var_decl $\rightarrow$ 'var' tipo ID mas_ids ';'

mas_ids $\rightarrow$ ';' ID mas_ids

mas_ids $\rightarrow \epsilon$

tipo $\rightarrow$ 'int'

tipo $\rightarrow$ 'float'

funcs $\rightarrow$ function funcs

funcs $\rightarrow \epsilon$

function $\rightarrow$ tipo ID '(' params_opt ')' ':' vars body ';'

params_opt $\rightarrow$ params

params_opt $\rightarrow \epsilon$

params $\rightarrow$ tipo ID mas_params

mas_params $\rightarrow$ ',' tipo ID mas_params

mas_params $\rightarrow \epsilon$

body $\rightarrow$ '{' statements '}'

statements $\rightarrow$ statement statements

statements $\rightarrow \epsilon$

statement $\rightarrow$ assign ';'

statement $\rightarrow$ condition

statement $\rightarrow$ cycle

statement $\rightarrow$ print

assign $\rightarrow$ ID '=' expression

print $\rightarrow$ 'print' '(' expression mas_expresiones ')' ';'

mas_expresiones $\rightarrow$ ',' expression mas_expresiones

mas_expresiones $\rightarrow \epsilon$

condition $\rightarrow$ 'if' '(' expression ')' body else_opt

else_opt $\rightarrow$ 'else' body

else_opt $\rightarrow \epsilon$

cycle $\rightarrow$ 'while' '(' expression ')' 'do' body

expression $\rightarrow$ exp expression_opcional

expression_opcional $\rightarrow$ relop exp

expression_opcional $\rightarrow \epsilon$

relop $\rightarrow$ '=='

relop $\rightarrow$ '!='

relop $\rightarrow$ '<'

relop $\rightarrow$ '<='

relop $\rightarrow$ '>'

relop $\rightarrow$ '>='

exp $\rightarrow$ termino exp_suma

exp_suma $\rightarrow$ '+' termino exp_suma
exp_suma $\rightarrow$ '-' termino exp_suma
exp_suma $\rightarrow \epsilon$

termino $\rightarrow$ factor exp_mult

exp_mult $\rightarrow$ '*' factor exp_mult
exp_mult $\rightarrow$ '/' factor exp_mult
exp_mult $\rightarrow \epsilon$

factor $\rightarrow$ '(' expresion ')'
factor $\rightarrow$ CTE_INT
factor $\rightarrow$ CTE_FLOAT

factor $\rightarrow$ CTE_STRING
factor $\rightarrow$ ID
factor $\rightarrow$ f_call

f_call $\rightarrow$ ID '(' args_opt ')'

args_opt $\rightarrow$ args
args_opt $\rightarrow \epsilon$

args $\rightarrow$ expresion mas_args

mas_args $\rightarrow$ ',' expresion mas_args
mas_args $\rightarrow \epsilon$

Análisis de Herramientas de Generación de Compiladores

Herramientas Investigadas:

1. Flex & Bison
2. ANTLR
3. JavaCC

1. Flex & Bison

- Son herramientas clásicas del entorno Unix/Linux, herederas de Lex & Yacc.
- Flex genera analizadores léxicos (scanners) basados en autómatas finitos deterministas (DFA).
- Bison genera analizadores sintácticos (parsers) basados en técnicas LALR(1).
- Están diseñadas para funcionar en lenguaje C, lo cual puede dificultar su integración con otros lenguajes modernos como Java.
- Utilizan la línea de comandos como interfaz principal.
- Son muy estables y ampliamente usadas en sistemas de tipo UNIX.
- Licencia: GPL.

2. ANTLR (Another Tool for Language Recognition)

- Herramienta moderna escrita en Java.
- Permite generar analizadores léxicos y sintácticos para múltiples lenguajes de destino como Java, Python, C#, JavaScript y Go.
- Combina la definición de lexer y parser en un solo archivo .g4, facilitando la organización de proyectos.
- Utiliza técnicas de análisis LL(*) predictivo, permitiendo gramáticas más naturales sin necesidad de eliminar recursión izquierda.
- Genera automáticamente componentes adicionales como listeners y visitors para recorrer árboles de sintaxis.
- Ampliamente utilizada en la industria y en el ámbito académico para diseñar lenguajes, interpretes y compiladores.
- Licencia: BSD.

3. JavaCC (Java Compiler Compiler)

- Generador de analizadores léxicos y sintácticos escrito completamente en Java.
- Produce código Java puro sin dependencias externas en tiempo de ejecución.
- Utiliza técnicas de análisis LL(k) recursivo descendente.
- Es más sencillo de integrar en proyectos Java pequeños o medianos que no requieran árboles de sintaxis complejos.

- Comparado con ANTLR, su popularidad ha disminuido, aunque sigue siendo útil para ciertos tipos de proyectos.
- Licencia: BSD.

Conclusión del análisis

ANTLR resulta la mejor opción para el desarrollo de Baby_Duck, debido a su perfecta integración con Java, su robustez, su facilidad de uso y su documentación actualizada.

Definición de la Gramática de BabyDuck

Componente léxico	Token	Expresión regular
Identificador	ID	[a-zA-Z_][a-zA-Z_0-9]*
Constante entera	CTE_INT	[0-9]+
Constante flotante	CTE_FLOAT	[0-9]+ '.' [0-9]+
Cadena de texto	CTE_STRING	"" (~["\\"])* ""
Palabras clave	KEYWORDS	program, main, end, var, int, float, void, print, if, else, while, do
Operadores	OPERATORS	+, -, *, /, ==, !=, <, <=, >, >=
Delimitadores	SYMBOLS	;;, :, ,, (,), {, }, [,]

```
badito — -zsh — 39x42

PROGRAM      : 'program' ;
MAIN         : 'main' ;
END          : 'end' ;
VAR          : 'var' ;
INT          : 'int' ;
FLOAT        : 'float' ;
VOID         : 'void' ;
PRINT        : 'print' ;
IF           : 'if' ;
ELSE         : 'else' ;
WHILE        : 'while' ;
DO           : 'do' ;

PLUS         : '+' ;
MINUS        : '-' ;
MULT         : '*' ;
DIV          : '/' ;
ASSIGN       : '=' ;
EQ           : '==' ;
NEQ          : '!=' ;
LT           : '<' ;
LEQ          : '<=' ;
GT           : '>' ;
GEQ          : '>=' ;

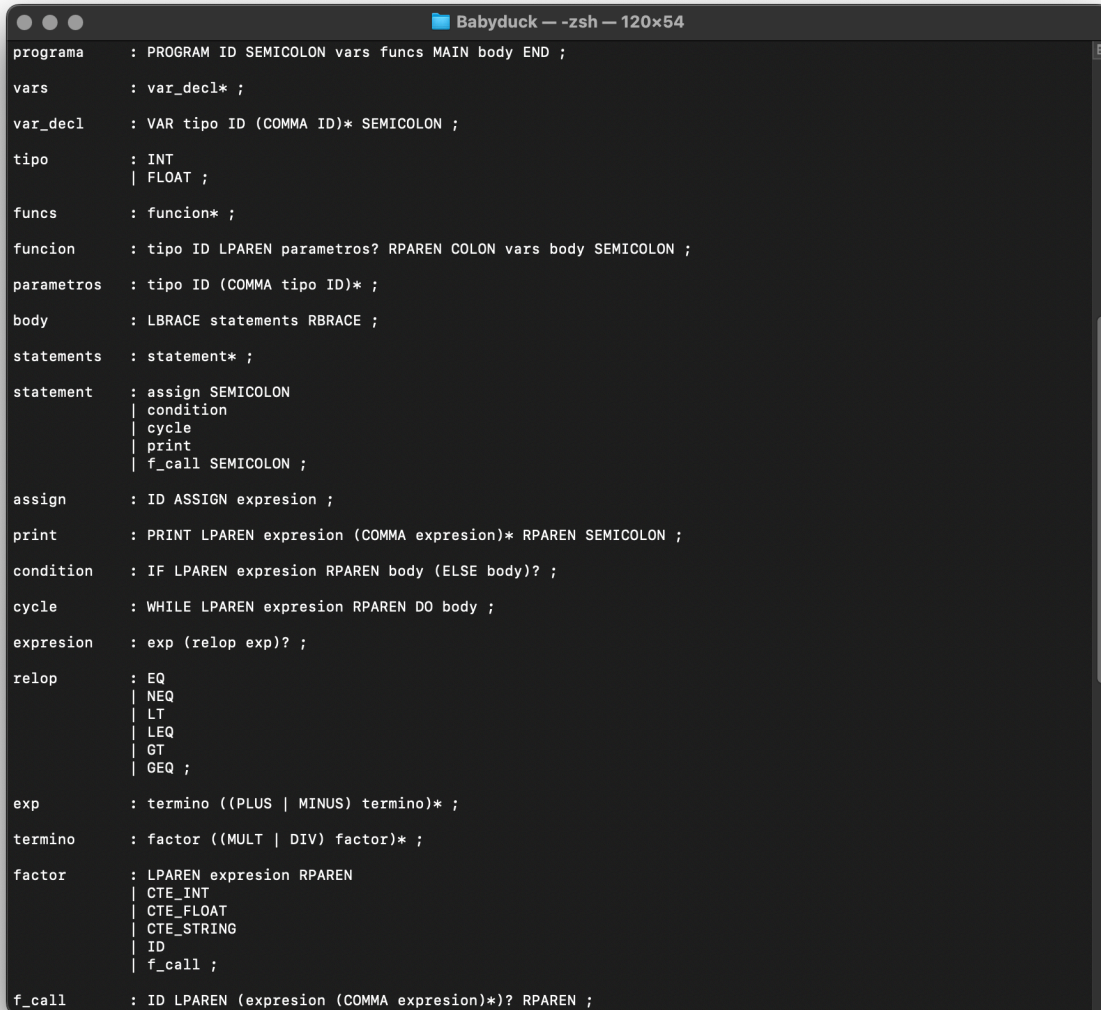
SEMICOLON    : ';' ;
COLON        : ':' ;
COMMA        : ',' ;
LPAREN       : '(' ;
RPAREN       : ')' ;
LBRACE       : '{' ;
RBRACE       : '}' ;
LBRACKET     : '[' ;
RBRACKET     : ']' ;

ID           : [a-zA-Z_][a-zA-Z_0-9]* ;
CTE_INT      : [0-9]+ ;
CTE_FLOAT    : [0-9]+ '.' [0-9]+ ;
CTE_STRING   : '"' (~["\\])* '"' ;

WS           : [ \t\r\n]+ -> skip ;
LINE_COMMENT : '//' ~[\r\n]* -> skip ;
```

Cada regla gramatical sigue la estructura:

- Nombre de la regla
- Secuencia de tokens y subreglas
- Uso de repeticiones (*), opciones (?) y agrupaciones



```
programa : PROGRAM ID SEMICOLON vars funcs MAIN body END ;
vars : var_decl* ;
var_decl : VAR tipo ID (COMMA ID)* SEMICOLON ;
tipo : INT
      | FLOAT ;
funcs : function* ;
function : tipo ID LPAREN parametros? RPAREN COLON vars body SEMICOLON ;
parametros : tipo ID (COMMA tipo ID)* ;
body : LBRACE statements RBRACE ;
statements : statement* ;
statement : assign SEMICOLON
           | condition
           | cycle
           | print
           | f_call SEMICOLON ;
assign : ID ASSIGN expression ;
print : PRINT LPAREN expression (COMMA expression)* RPAREN SEMICOLON ;
condition : IF LPAREN expression RPAREN body (ELSE body)? ;
cycle : WHILE LPAREN expression RPAREN DO body ;
expression : exp (relop exp)? ;
relop : EQ
       | NEQ
       | LT
       | LEQ
       | GT
       | GEQ ;
exp : termino ((PLUS | MINUS) termino)* ;
termino : factor ((MULT | DIV) factor)* ;
factor : LPAREN expression RPAREN
        | CTE_INT
        | CTE_FLOAT
        | CTE_STRING
        | ID
        | f_call ;
f_call : ID LPAREN (expression (COMMA expression))* RPAREN ;
```


Organización de los Formatos

- Las expresiones regulares para el léxico fueron definidas en la sección de Lexer Rules del archivo .g4.
- Las reglas gramaticales fueron estructuradas en la sección de Parser Rules del mismo archivo.
- Se utilizó una combinación de la notación BNF y EBNF simplificada (propia de ANTLR), que permite expresar repeticiones, opcionales y alternativas de manera compacta.

Caso	Descripción	Entrada de prueba	Resultado
TC1	Programa mínimo válido	<pre>program hola; main { } end</pre>	Aceptado
TC2	Declaración de variables múltiples	<pre>program varTest; var int x, y, z; main { x = 1; y = 2; z = x + y; print(z); } end</pre>	Aceptado
TC3	Función sin parámetros	<pre>program funcSinParams; int suma() : var int a; { a = 1; print(a); }; main { print(1); } end</pre>	Aceptado
TC4	Función con parámetros	<pre>program funcParams; int mult(int x, float y) : var float res; { res = x * y; print(res); }; main { print(2); } end</pre>	Aceptado

TC5	Uso de while	<pre> program ciclo; var int i; main { i = 3; while (i > 0) do { print(i); i = i - 1; } } end </pre>	Aceptado
TC6	Condicional if-else	<pre> program decision; var int a; main { a = 5; if (a < 10) { print("Menor que 10"); } else { print("Mayor o igual que 10"); } } end </pre>	Aceptado
TC7	Error léxico	<pre> program 123Invalid; main { } end </pre>	Error (identificador inválido 123Invalid)
TC8	Error sintáctico (falta ; en asignación)	<pre> program errorSintaxis; var int x; main { x = 10 print(x); } end </pre>	Error (falta punto y coma después de x = 10)
TC9	Llamada a función	<pre> program llamado; int cuadrado(int x) : var int r; { r = x * x; print(r); }; main { cuadrado(5); } end </pre>	Aceptado

TC10	Expresión compuesta	<pre> program expr; var float resultado; main { resultado = (3 + 5) * 2.5; print(resultado); } end </pre>	Aceptado
-------------	------------------------	---	----------

```

Babyduck — -zsh — 101x13
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc1.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc2.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc3.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc4.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc5.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc6.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc7.bd
line 1:8 extraneous input '123' expecting ID
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc8.bd
line 5:4 missing ';' at 'print'
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc9.bd
badito@Gabriels-MacBook-Pro Babyduck % java -cp ".:antlr-4.13.2-complete.jar" Main tests/tc10.bd
badito@Gabriels-MacBook-Pro Babyduck %

```